



Xeus-Haskell

Interactive Haskell Computing in the Browser

Masaya Taniguchi · RIKEN AIP

Haskell 2026 · ICFP 2026

Demo <https://jupyter-xeus.github.io/xeus-haskell>

GitHub <https://github.com/jupyter-xeus/xeus-haskell>

Origin

Research

Field · Formal grammar researcher

Method · Categorical formalization

Medium · Executable Haskell

Goal · Reproducible experiments

From need to project

Need · A small Haskell laboratory

Build · A browser kernel

Community · DataHaskell → Jupyter-Xeus

GSoC · 2026 mentor



A research need became open-source infrastructure.

Installation Barrier

I wanted interactive notes for **Category Theory for Programmers**.

```
$ brew install ghcup python3 zeromq libmagic cairo pkg-config pango
$ pip3 install jupyter --user
$ ghcup install ghc recommended
$ ghcup install cabal recommended
$ cabal install ihaskell
$ ihaskell install --prefix="$HOME/.local/"
```

I do not have the brain for this on a Sunday.

Motivation

Python learners open Colab and start experimenting.

Haskell should be equally immediate.

No notebook server. No local toolchain. Just a URL.

Use Cases

THREE BARRIERS

Teaching	Setup becomes the first assignment.
Hosting	Thirty students contend for one server.
Live demo	Untrusted code runs on your infrastructure.

OUR SOLUTION

Static files	Users open a browser; providers serve static files.
--------------	---

Demo Track

The accepted demonstration focuses on the MicroHs kernel.

Kernel	Backend	JupyterLite
xhaskell-mhs	MicroHs	Yes

Browser execution · persistence · Jupyter · rich display

Demo: <https://jupyter-xeus.github.io/xeus-haskell>

MicroHs Demo

CELL 1 · DECLARATIONS

```
data Tree a = Leaf a | Branch (Tree a) (Tree a)
size (Leaf _) = 1
size (Branch l r) = size l + size r
```

CELL 2 · EXPRESSION

```
size (Branch (Leaf 'a') (Leaf 'b'))
```

OUTPUT · 2

One persistent browser session

MicroHs

Extended Haskell subset with a combinator runtime.

- Minimal-dependency runtime, including microcontrollers
- Self-hosting compiler
- Bootstrap from combinators with a C compiler
- JavaScript and Wasm targets through Emscripten

Source: <https://microhs.org/>

Bonus: GHC Wasm



Everything so far is the accepted MicroHs demonstration.

Kernel	Backend	JupyterLite
xhaskell-ghc	GHC Wasm	Yes

- GHC runs entirely on the client.
- One session preserves declarations, imports, IO state, and `it`.
- Notebook users get GHC/GHCi language behavior.

New work beyond the submitted demonstration.

GHC in the Browser

Frontend live-coding via **GHCi** · April 2025

- `-fghci-browser`: host GHCi → browser Wasm
- **ghc-in-browser**: GHC compiled to Wasm
- Client-side compiler, type checker, and bytecode interpreter
- In-browser GHC filesystem
- No server-side compiler

Source: <https://www.tweag.io/blog/2025-04-17-wasm-ghci-browser/>

Source: <https://haskell-wasm.github.io/ghc-in-browser/>

GHC Demo



```
In [1]: :set -XTypeApplications
```

```
In [2]: :t fmap @Maybe
```

```
      fmap @Maybe :: (a -> b) -> Maybe a -> Maybe b
```

```
In [3]: fmap @Maybe (+1) (Just 41)
```

```
Out[3]: Just 42
```

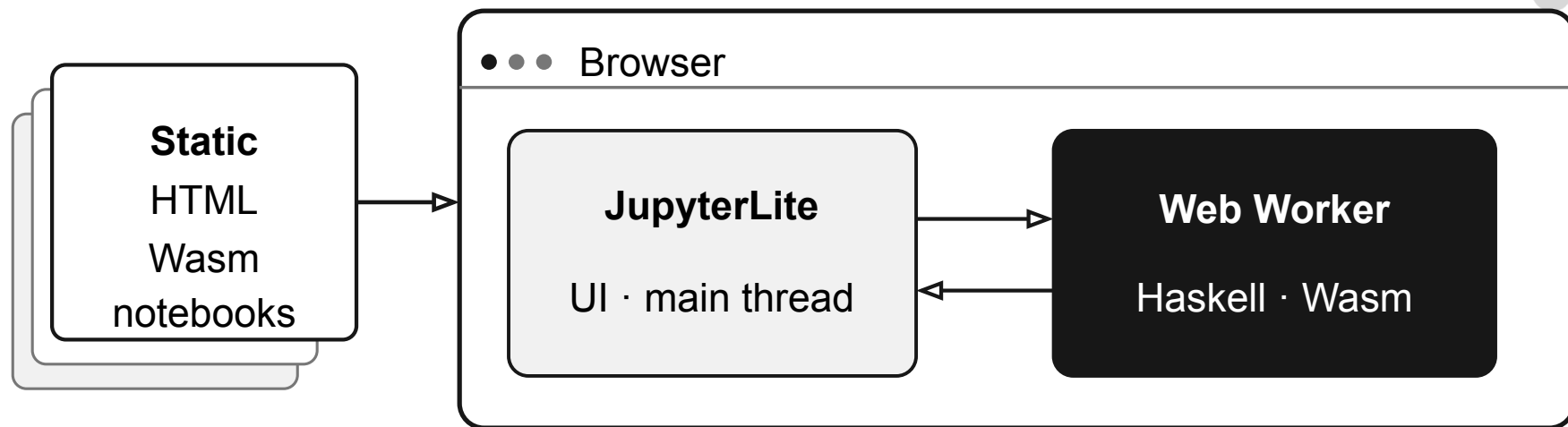
GSoC 2026: persistent GHC · JSPI bridge · WASI assets · browser tests

Architecture and implementation: Masaya Taniguchi

JupyterLite



JupyterLab as a static, serverless web application.

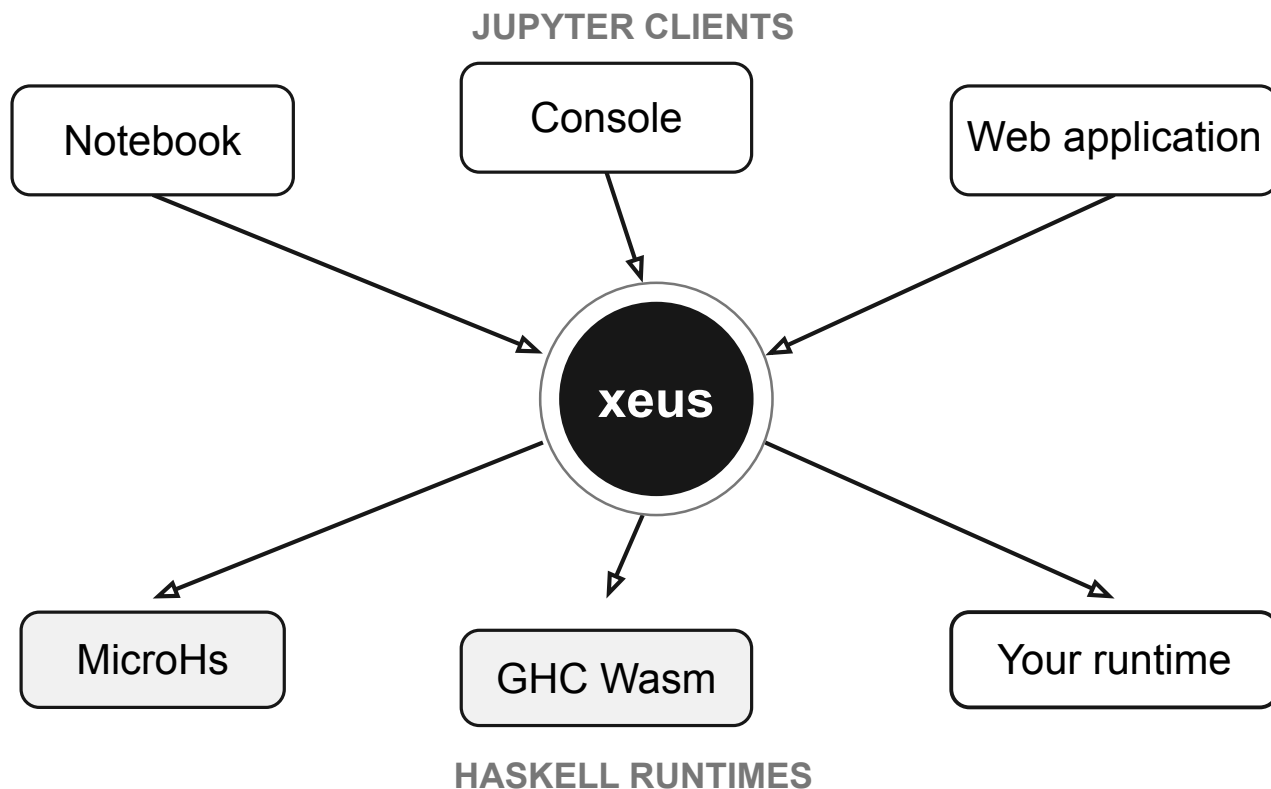


Source: <https://jupyterlite.readthedocs.io/en/stable/>

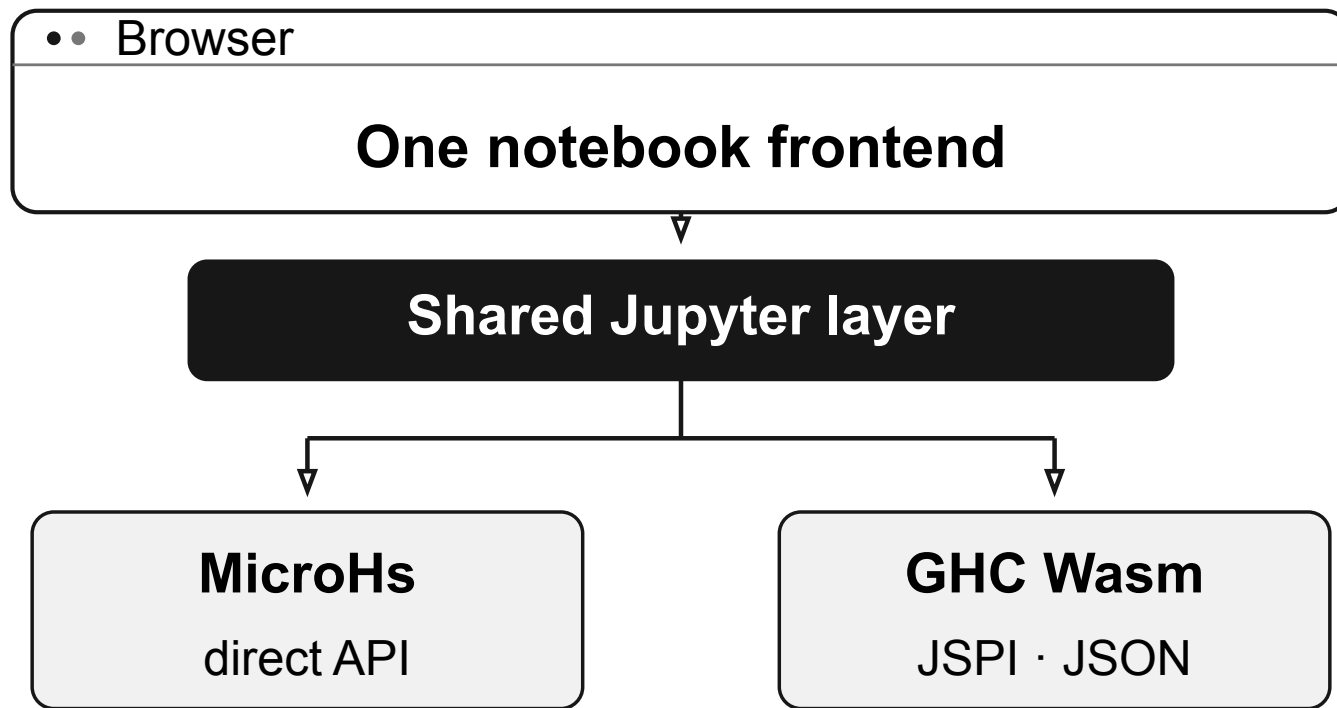
Xeus



xeus is a C++ implementation of the Jupyter kernel protocol.



System Architecture



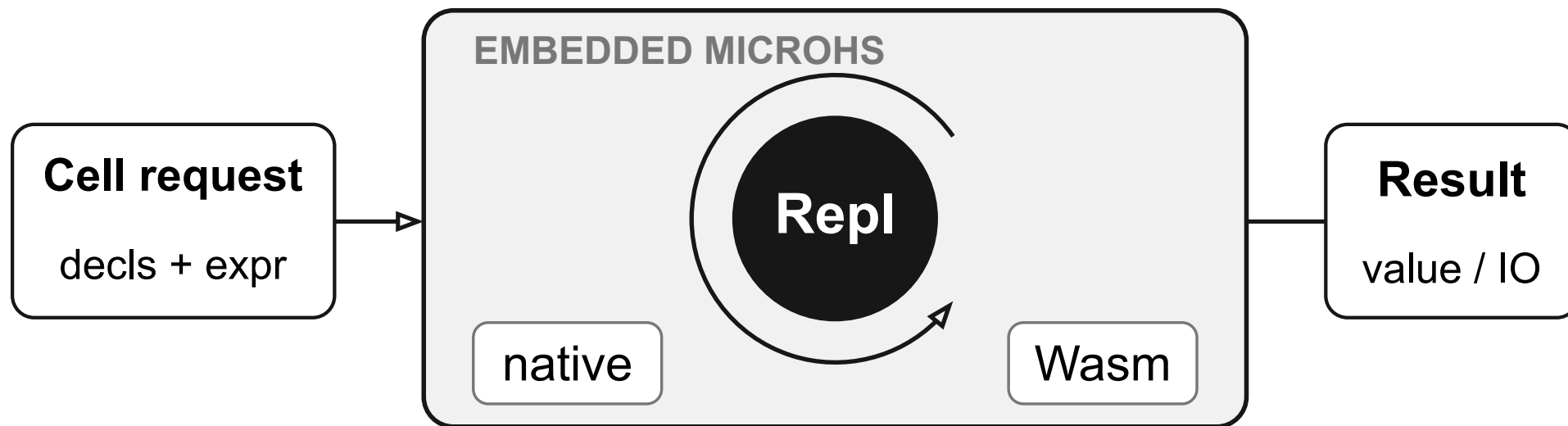
Both demos share one notebook frontend.

Backend Trade-offs

	MicroHs	GHC Wasm
Goal	Small and portable Haskell	GHC/GHCi compatibility
Runtime	Embedded compiler API	Persistent browser GHC session
Where	Native and browser	Browser

MicroHs: portability · GHC Wasm: compatibility

MicroHs Backend



Direct API · persistent state · native and Wasm

MicroHs Expansion

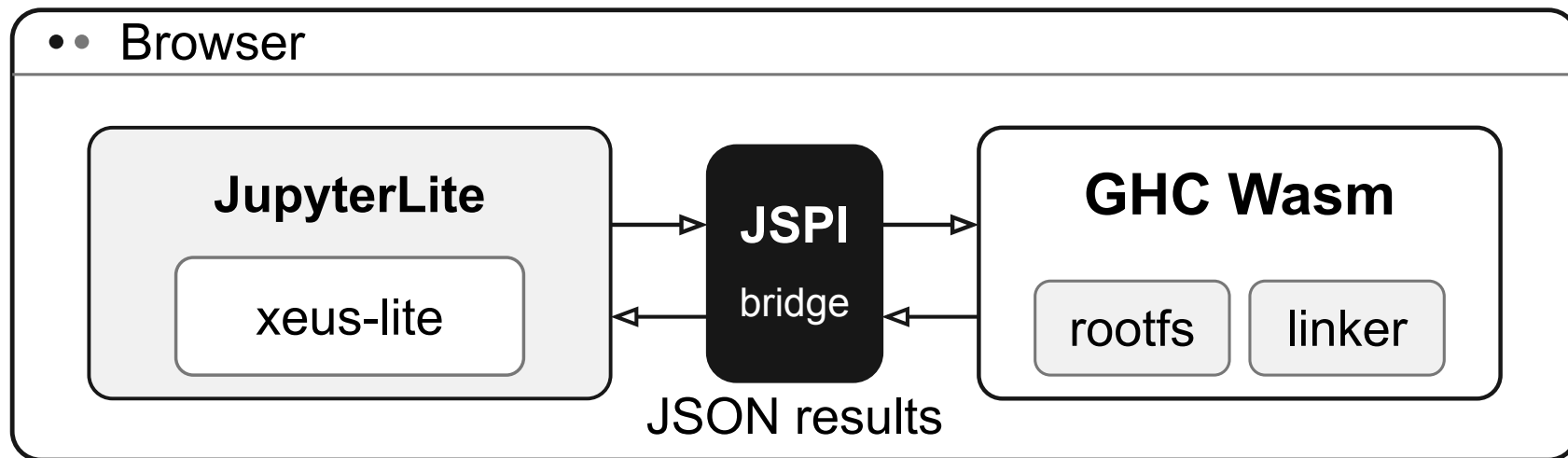
Cell

```
square x = x * x  
map square [1..5]
```

Generated execution module

```
module Inline where  
import Prelude  
import System.IO.PrintOrRun  
square x = x * x  
runResult = _printOrRun (map square [1..5])
```

GHC Wasm Backend



Rootfs and linker assets live in an in-memory filesystem.

One browser session preserves GHC state across cells.

GHCi Dispatch

Cell

```
square x = x * x  
map square [1..5]
```

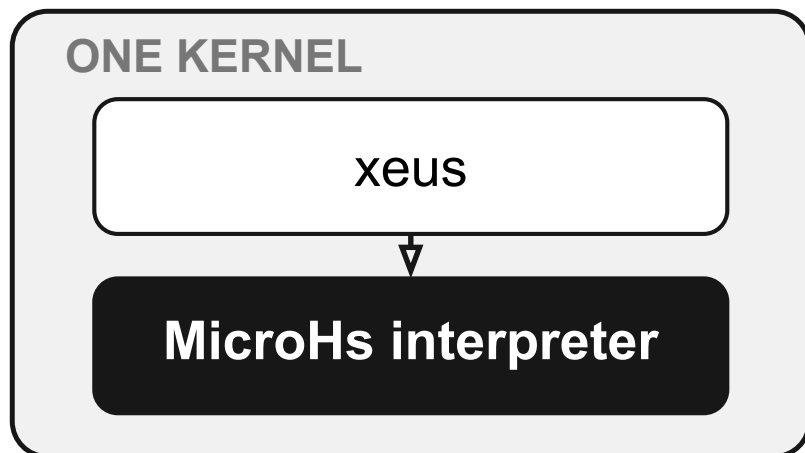
Executed in one persistent GHC session

```
runDecls "square x = x * x"  
execStmt "map square [1..5]" execOptions
```

```
xhaskellGhcInteractivePrint value =  
    putStrLn resultMarker >> print value
```

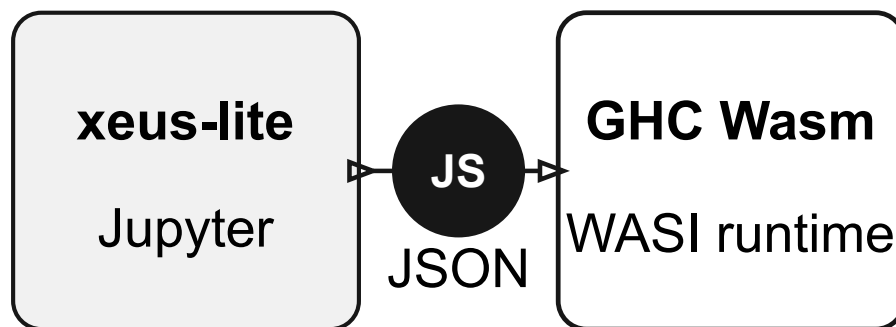
Integration Models

Monolithic



one binary · native streams

Modular



independent runtime · JSON boundary

Two runtimes expose two reusable integration patterns.

Interactive Semantics

Case	Example	Judgment	State
Declaration	square $x=x*x$	$\Gamma_n \vdash d \Rightarrow \Gamma_{n+1}$	Γ grows.
Expression	square 5	$\Gamma_n \vdash e : \tau \cdot e \Downarrow v$	Γ stays.
IO action	print \$ square 5	$\Gamma_n \vdash m : \text{IO } \tau \cdot \langle m, \sigma \rangle \Downarrow \langle v, \sigma' \rangle$	σ may change.

Read left to right: cell · typing judgment · state transition

Γ : typed environment · σ : effect state · $\Downarrow$: evaluates to

Haskell separates environment growth, pure evaluation, and explicit effects.

Rich Display

```
import XHaskell.Display
```

```
putStr $ show $ DisplayData "text/latex"  
  "\\[\\int_0^1 x^2 dx = \\frac{1}{3}\\]"
```

$$\int_0^1 x^2 dx = \frac{1}{3}$$

A small framed MIME protocol becomes Jupyter display_data.

```
<STX><MIME type><US><content><ETX>
```

Acknowledgements



- Google Summer of Code 2026 · Haskell.org
 - Arman Sanjay Choudhary for early GHC/Wasm exploration
- ACM SIGPLAN Professional Activities Committee
- Haskell 2026 organizers and community
- JSPS Grant-in-Aid for Early-Career Scientists (24K16077)
- HUMAI Foundation (A)

Conclusion

Haskell notebooks can run entirely in the browser.

No notebook server. No local toolchain. Just a URL.

One Jupyter layer · two complementary backends

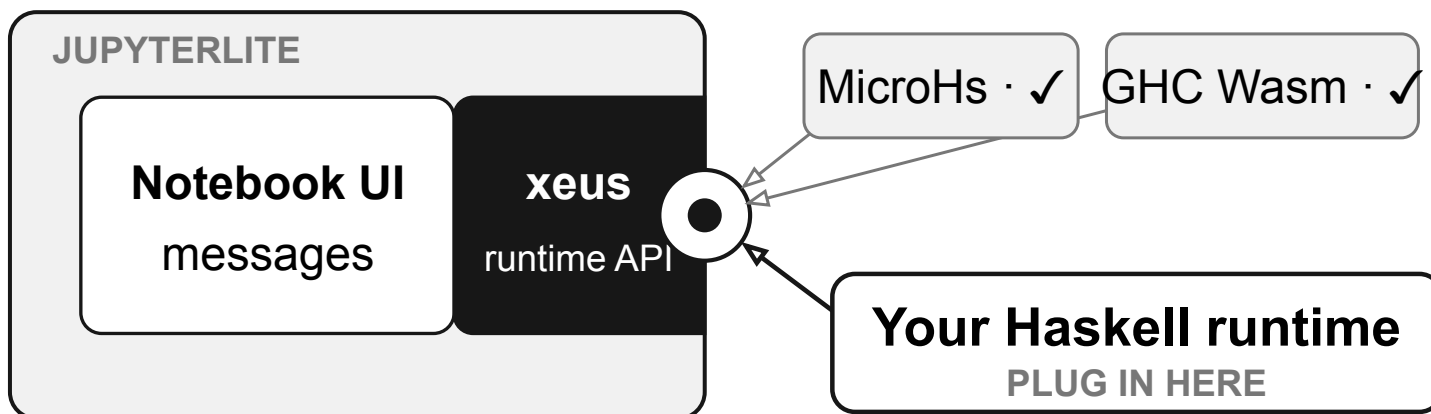
MicroHs for portability · GHC Wasm for compatibility

Demo: <https://jupyter-xeus.github.io/xeus-haskell>

Source: <https://github.com/jupyter-xeus/xeus-haskell>

Collaboration

Building a Haskell implementation? Let us bring it to Jupyter.

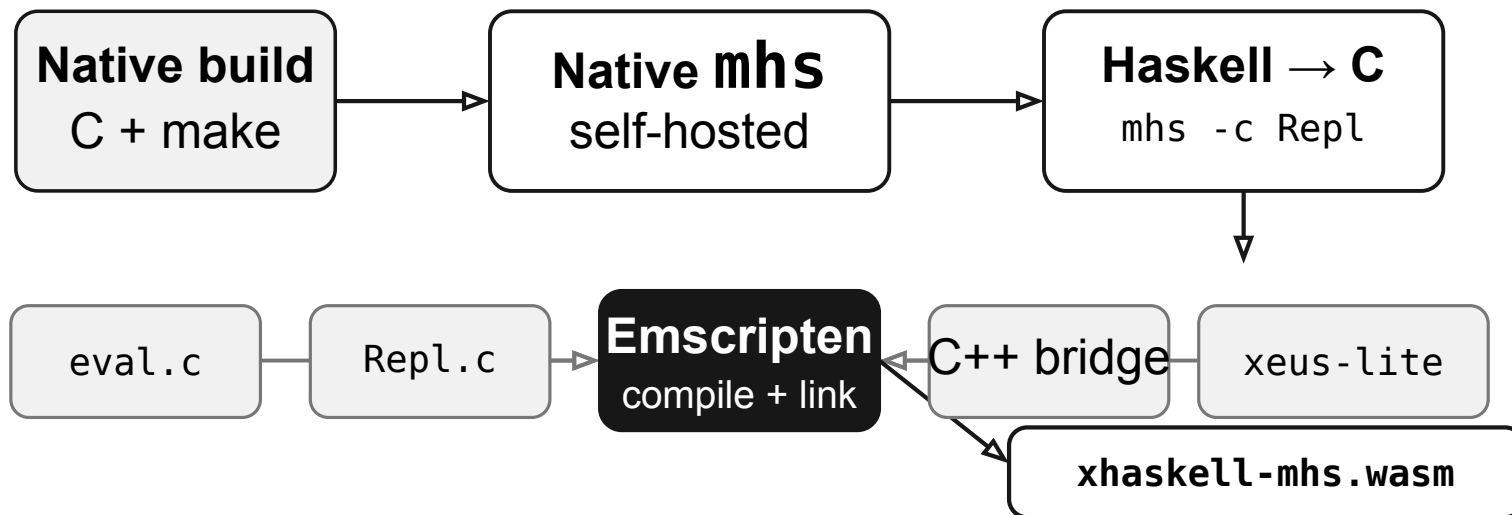


You bring the runtime; xeus brings Jupyter.

Collaborate: <https://github.com/jupyter-xeus/xeus-haskell>

MicroHs Build

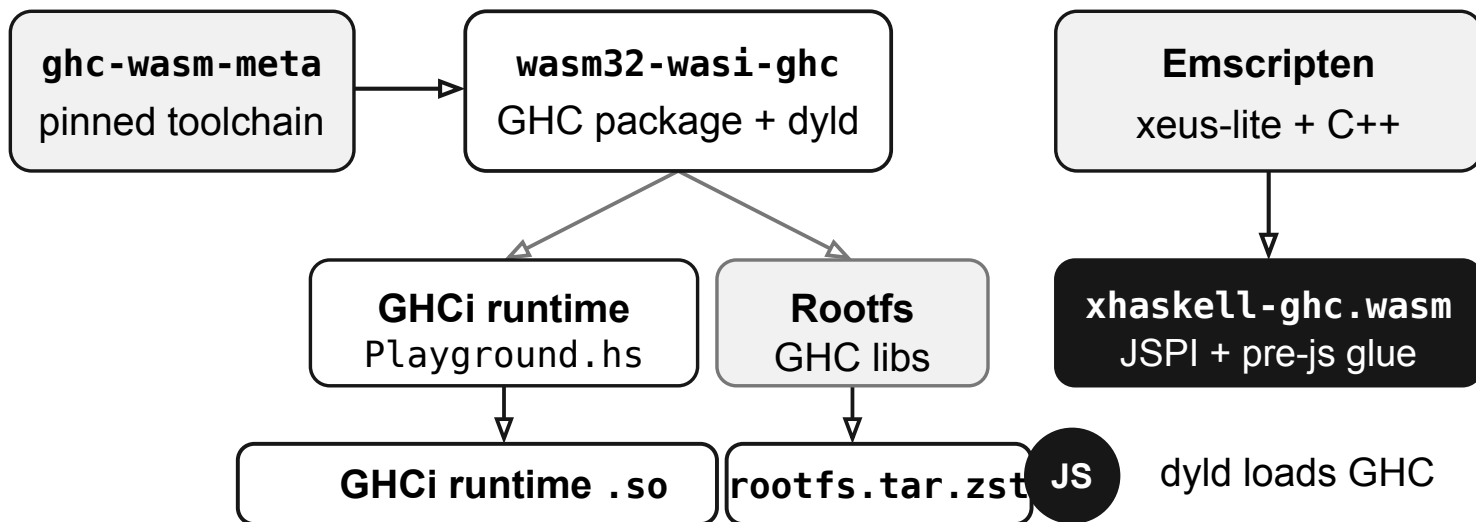
APPENDIX · BUILD RECIPE



xeus calls MicroHs directly as an in-process library.

GHC Wasm Build

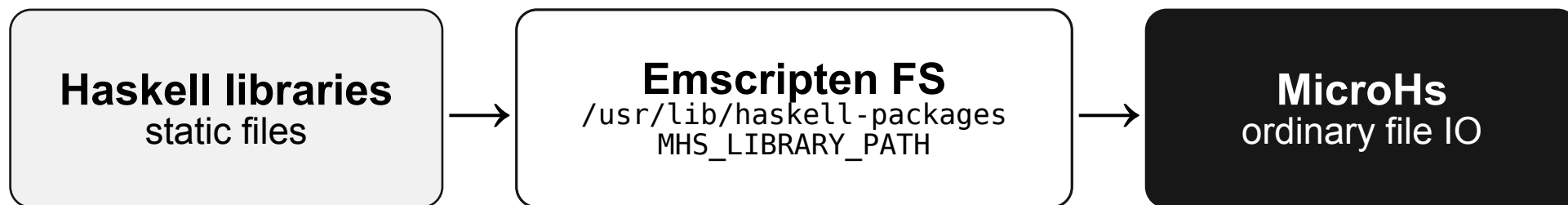
APPENDIX · BUILD RECIPE



Bundled together; loaded at runtime; not linked into one Wasm module.

Library Bundling

APPENDIX · EMSCRIPTEN FS



Route	Mechanism	Result
JupyterLite	XeusAddon.mounts	Files enter MEMFS before the worker starts.
Standalone	--preload-file dir@/path	A separate .data file is fetched.
Single script	--embed-file dir@/path	Files are embedded in generated JavaScript.

The compiler sees files; deployment serves static assets.

Logo Licenses

APPENDIX · ATTRIBUTION

Mark · reuse basis	Reference
Haskell · public domain	https://commons.wikimedia.org/wiki/File:Haskell-Logo.svg
WebAssembly · CC0 1.0	https://commons.wikimedia.org/wiki/File:WebAssembly_Logo.svg
Jupyter · BSD + trademark policy	https://jupyter.org/governance/trademarks.html
xeus · Apache-2.0	https://github.com/jupyter-xeus/xeus-haskell
GSoC · official participation use	https://developers.google.com/open-source/gsoc/resources/brand_guidelines
ICFP 2026 · official event mark	https://icfp26.sigplan.org/

The -bg.png derivatives only change opacity to 40%; trademark rules apply; no endorsement is implied.

No DataHaskell or GHC logo: no explicit reusable artwork license was identified.